Mạch - Logic Lab

Báo cáo Bài tập lớn 1: Tìm kiếm

Nhập môn Trí tuệ nhân tạo, học kỳ 1 năm học 2026-2027. Phiên bản kỹ thuật ngày 25/10/2026.

Tóm tắt

Ứng dụng hiện thực hai bài toán Logic Puzzles: Pipes và Light Up (Akari). Mỗi bài toán được giải bằng Depth-First Search (DFS) và Greedy Best-First Search (Greedy), dùng chung biểu diễn trạng thái, thứ tự chọn biến và quy tắc cắt nhánh. Phần tìm kiếm và kiểm tra luật được viết bằng Python 3; HTML/CSS/JavaScript đảm nhiệm tương tác và minh họa.

Sản phẩm cung cấp 8 màn chơi, demo từng trạng thái được xét, gợi ý theo các lựa chọn hiện tại, sửa lựa chọn mâu thuẫn, hoàn tác và lưu tiến độ. Thực nghiệm mở rộng dùng 26 input, 2 thuật toán và 7 lần đo thời gian cho mỗi tổ hợp. Cả 364 lượt đo thời gian và 52 lượt đo bộ nhớ đều tìm được nghiệm được kiểm tra lại theo luật. Kết quả cho thấy hiệu quả của heuristic phụ thuộc cấu trúc input; Greedy không luôn nhanh hơn DFS.

1. Phạm vi và đối chiếu yêu cầu

Theo đề BTL1, nhóm cần chọn hai Logic Puzzles, hiện thực Python 3, mỗi bài có một thuật toán tìm kiếm mù và một thuật toán có heuristic; nên có demo trực quan từng bước, đồng thời phải báo cáo thời gian, bộ nhớ và giải thích kết quả. Dự án chọn DFS cho tìm kiếm mù và Greedy cho tìm kiếm có heuristic.

Yêu cầu	Hiện thực và bằng chứng
Hai bài toán	Pipes và Light Up, mô-đun riêng trong logic_lab/games
Python 3	Bộ máy tìm kiếm, luật, API, CLI và benchmark
Tìm kiếm mù	DFS dùng stack, không dùng heuristic để chọn nhánh
Tìm kiếm heuristic	Greedy dùng heap với điểm h, không dùng g + h
Input phù hợp	8 màn chơi và 18 input sinh thêm bằng seed cố định; hỗ trợ JSON riêng
Demo trực quan	Lùi/tiến, thanh trượt, tự chạy, xem bảng nghiệm
Đo thời gian và bộ nhớ	benchmark.json, benchmark.csv và bảng thực nghiệm kèm báo cáo
Giải thích kết quả	Mô hình, heuristic, phân tích thực nghiệm và giới hạn trong báo cáo

Đề không bắt buộc A*. Việc chọn Greedy đáp ứng nhóm giải thuật heuristic nhưng không có bảo đảm tối ưu đường đi hoặc số thao tác của người chơi. Bản cài đặt không hỗ trợ Pipes nối xuyên biên.

2. Luật của hai bài toán

2.1. Pipes

Mỗi ô chứa một mảnh ống. Người chơi xoay từng mảnh để mọi đầu nối khớp với ô kề cạnh, không chĩa ra ngoài bảng. Toàn bộ ống phải nằm trong một mạng liên thông và không có vòng kín. Đây là luật được đối chiếu với trang Pipes [1].

Xét đồ thị có một đỉnh cho mỗi ô và một cạnh cho mỗi cặp đầu nối khớp. Bảng thắng khi không có đầu hở, đồ thị liên thông và không chu trình. Với n đỉnh, điều kiện cạnh tương đương của một cây là có n - 1 cạnh; bộ kiểm tra vẫn kiểm tra đồng thời độ khớp, liên thông và số chu trình để phản hồi được cả bảng sai.

2.2. Light Up

Bảng gồm ô trắng và ô đen. Một đèn chỉ đặt ở ô trắng và chiếu theo hàng/cột đến biên hoặc ô đen. Tất cả ô trắng phải sáng; hai đèn không được chiếu trực tiếp vào nhau. Ô đen có số k phải có đúng k đèn kề cạnh, không tính đường chéo. Ô đen không số chỉ chặn ánh sáng [2].

Dấu × là ghi chú của người chơi về việc không đặt đèn, không phải vật cản ánh sáng. Hai tia sáng được giao nhau. Ứng dụng kiểm tra lời giải theo luật, chấp nhận nhiều nghiệm thay vì so với một đáp án lưu sẵn.

3. Mô hình hóa Pipes

3.1. Biểu diễn trạng thái

Các đầu nối dùng bitmask: trên = 1, phải = 2, dưới = 4, trái = 8. Ví dụ mask 6 biểu diễn nối phải và dưới. Xoay 90 độ theo chiều kim đồng hồ là quay vòng bốn bit. Chỉ giữ các hướng phân biệt: ống thẳng có 2 hướng, ống chữ thập có 1 hướng.

Trạng thái tìm kiếm là tuple gồm n hướng cuối cùng đang được gán. Giá trị 0 nghĩa là chưa gán, không có nghĩa ô trống. Với bài gốc, trạng thái đầu là toàn 0. Khi giữ các lựa chọn hiện tại, chỉ các ô người chơi đã chỉnh mới được gán cố định. Hướng trên bảng chơi ở ô chưa chỉnh chỉ dùng để hiển thị.

Một hành động search chọn ô chưa gán đầu tiên theo thứ tự hàng/cột rồi gán một hướng. Các trạng thái là bất biến; sinh con bằng tuple mới. Thứ tự chọn biến cố định giúp mỗi phép gán có đúng một đường đi từ gốc và không cần một tập visited cho các hoán vị thao tác.

3.2. Ràng buộc và cắt nhánh

Miền hướng của mỗi ô được loại bỏ các hướng chĩa ra ngoài bảng. Tại mỗi trạng thái, solver loại các hướng không khớp với hàng xóm đã gán. Nếu miền bất kỳ rỗng thì cắt nhánh. Union-find phát hiện chu trình trong các cạnh giữa những ô đã gán. Cuối cùng, duyệt đồ thị gồm các cạnh vẫn có thể xuất hiện để kiểm tra khả năng liên thông.

Các điều kiện này là cần thiết: khi một biến đã gán, hậu duệ không đổi hướng đó; do đó một cạnh sai hoặc chu trình đã hình thành không thể tự biến mất. Nếu ngay cả đồ thị chứa mọi cạnh còn khả thi cũng không liên thông thì không một tập con cạnh nào có thể nối toàn bộ bảng. Cắt nhánh chỉ loại trạng thái không có cách hoàn thiện.

3.3. Điều kiện đích và heuristic

Đích yêu cầu mọi ô đã được gán và bộ kiểm tra bảng hoàn chỉnh trả về đúng luật. Greedy dùng:

h(s) = sum(1 + 0.25 * (len(D_i) - 1)) với các ô i chưa gán.

D_i là miền hướng còn lại sau kiểm tra cục bộ. Số ô chưa gán thể hiện khối lượng quyết định còn lại; phần cộng thêm phạt sự không chắc chắn. Điểm thấp ưu tiên trạng thái đã gán nhiều ô và có ít lựa chọn còn lại. Đây là hàm xếp hạng; không khẳng định admissible hay consistent cho một bài toán tối ưu chi phí thao tác.

Một lần gán hướng trong search có thể cần 1-3 lần xoay 90 độ trên giao diện. Demo đang minh họa tìm phép gán thỏa ràng buộc, không tìm chuỗi xoay ít bước nhất.

4. Mô hình hóa Light Up

4.1. Biểu diễn và tiền xử lý

Mỗi ô trắng là một biến: -1 = chưa quyết định, 0 = không có đèn, 1 = có đèn. Ô đen không phải biến. Tọa độ ngoài API dùng chỉ số hàng * số_cột + cột, bắt đầu từ 0; solver có ánh xạ riêng sang chỉ số các ô trắng.

Trước search, chương trình tính tập ô nhìn thấy từ mỗi ô trắng, gồm cả chính nó, và tập hàng xóm trắng của từng ô số. Các tập này được tái sử dụng khi tính vùng sáng, xung đột và ứng viên đặt đèn. Đèn người chơi trở thành giá trị 1, dấu × thành 0, các ô còn lại là -1.

4.2. Sinh trạng thái và cắt nhánh

Chọn ô -1 đầu tiên theo hàng/cột; thử 0 trước, sau đó 1 nếu ô đó còn là ứng viên hợp lệ. DFS và Greedy có cùng thứ tự sinh con. Một ứng viên phải chưa quyết định và chưa được đèn khác chiếu tới. Ô số đã đủ đèn loại các ô kề cạnh còn lại khỏi tập ứng viên.

Cắt nhánh nếu có đèn nhìn nhau, ô số vượt chỉ tiêu, số đèn đã có cộng số ứng viên kề cạnh không đủ chỉ tiêu, hoặc một ô chưa sáng không còn ứng viên nào chiếu tới được. Các điều kiện chỉ loại khả năng không thể hoàn thành: một đèn mới không thể xóa xung đột cũ; một mục tiêu không còn vị trí khả thi thì không thể được thỏa trong hậu duệ.

4.3. Điều kiện đích và heuristic

Đích yêu cầu mọi ô trắng sáng, không đèn nào nhìn thấy đèn khác và mọi ô số chính xác. Có thể đạt đích trước khi quyết định toàn bộ biến; các biến -1 còn lại được hiểu là không đặt thêm đèn. Điều này vẫn là một lời giải đầy đủ theo luật.

h(s) = số ô trắng chưa sáng + tổng số đèn còn thiếu quanh các ô số.

Heuristic khuyến khích chiếu sáng nhiều ô và thỏa chỉ tiêu quanh ô đen. Một đèn có thể thỏa nhiều mục tiêu nên h có thể đếm chồng lợi ích; nó không phải cận dưới số đèn cần thêm. Greedy có thể đi vào nhánh h nhỏ nhưng gây nhiều bế tắc về sau, điều được quan sát trong thực nghiệm.

5. Bộ máy DFS và Greedy

5.1. DFS

DFS dùng stack. Để tôn trọng thứ tự sinh con, các con được đẩy lên stack theo thứ tự ngược. Khi không ghi trace, DFS không gọi heuristic. Khi ghi trace, h chỉ là số liệu minh họa, không ảnh hưởng lựa chọn trạng thái tiếp theo.

5.2. Greedy Best-First Search

Greedy dùng min-heap với khóa (h, thứ_tự_thêm). Khi bằng h, trạng thái được thêm trước được xét trước. Cơ chế này vừa tái lập được kết quả vừa tránh so sánh các đối tượng trạng thái để phá hòa. Greedy chỉ xếp theo h; không có chi phí g và không phải A*.

5.3. Giả mã dùng chung

```
analyze(initial); đưa gốc khả thi vào frontier
while frontier không rỗng:
    nếu hết ngân sách: trả limit
    lấy trạng thái theo stack (DFS) hoặc h nhỏ nhất (Greedy)
    lưu trace nếu còn chỗ; kiểm tra đích
    nếu là đích: trả solved cùng lời giải
    sinh các con theo cùng thứ tự biến và thứ tự giá trị
    analyze từng con; chỉ đưa con khả thi vào frontier
trả unsat
```

5.4. Tính đúng, đầy đủ và tối ưu

Mỗi bước gán thêm một biến, không bỏ sót giá trị hợp lệ của biến được chọn. Hai miền là hữu hạn: tối đa 4 hướng mỗi ô Pipes và 2 quyết định mỗi ô trắng Light Up. Các quy tắc cắt nhánh là điều kiện cần, và mọi nghiệm cuối được kiểm tra bằng luật. Vì vậy, khi không giới hạn tài nguyên, cả hai thuật toán đầy đủ trên không gian hữu hạn này.

Phiên bản web có timeout và số trạng thái tối đa. Chỉ frontier cạn sau khi duyệt hết mới chứng minh vô nghiệm. limit nghĩa là chưa biết; không được diễn giải là vô nghiệm. Cả DFS lẫn Greedy không bảo đảm ít thao tác giao diện nhất, ít đèn nhất hoặc ít nước sửa nhất.

5.5. Độ phức tạp

Gọi n là số biến, b là số nhánh tối đa và m không vượt n là độ sâu. Không gian gán thô là O(4^n) cho Pipes và O(2^n) cho Light Up. DFS/Greedy có thể phải xét số trạng thái theo hàm mũ trong trường hợp xấu; cắt nhánh làm giảm số trạng thái thực tế nhưng không thay đổi kết luận xấu nhất.

DFS giữ O(bm) node chờ; mỗi node chứa tuple và miền ứng viên kích thước O(n), nên bộ nhớ cài đặt có thêm thừa số n so với cách chỉ đếm node. Greedy có thể giữ O(b^m) node frontier và có chi phí push/pop heap O(log F), với F là kích thước frontier. Cả hai còn lưu cấu trúc tiền xử lý của bài toán. Light Up lưu các tập nhìn thấy nên có thể tốn O(n^2) trong trường hợp xấu. Không thể coi frontier_peak là tổng bộ nhớ chương trình.

6. Input và cách xây dựng

6.1. Tám màn tương tác

Pipes có kích thước 3, 4, 5, 6 với seed 11, 27, 43, 71. Bộ sinh tạo một cây khung bằng duyệt ngẫu nhiên rồi xoay các mảnh ống. Cây khung trước khi xoay là một nghiệm chứng kiến; hướng ban đầu luôn được làm khác nghiệm đó.

Light Up có ví dụ 3x3 với ô số 0 ở giữa; các màn 5, 6, 7 dùng seed 19, 31, 57. Bộ sinh đặt ô đen với xác suất 0.24, chọn đèn từ tập ô chưa sáng đến khi phủ hết ô trắng, rồi gán số cho một phần ô đen theo các đèn đã tạo. Chọn đèn từ ô chưa sáng tránh việc đèn mới nhìn thấy đèn cũ vì quan hệ nhìn thấy đối xứng. Các màn sinh do đó có nghiệm chứng kiến.

Nghiệm chứng kiến chỉ dùng kiểm chứng dữ liệu. Solver không đọc nghiệm này, API loại trường nội bộ _witness khi gửi màn cho trình duyệt. Các bộ sinh không chứng minh nghiệm duy nhất. Nhãn độ khó trên giao diện là hướng dẫn trải nghiệm, không phải phân lớp thực nghiệm có bảo đảm.

6.2. Bộ mở rộng và JSON riêng

Thực nghiệm mở rộng giữ 8 màn trên, thêm 18 input: 2 game x 3 kích thước (4, 6, 8) x 3 seed liên tiếp (101, 102, 103). Các seed không được chọn theo thuật toán thắng. Tổng cộng 26 input, chia đều 13 input mỗi game.

CLI và benchmark đọc JSON có game, rows, cols và tiles hoặc board; không yêu cầu đáp án. Bộ kiểm tra cấu trúc từ chối dữ liệu sai kiểu, ô không hợp lệ, bảng không chữ nhật hoặc id trùng. Input mâu thuẫn về luật vẫn được chấp nhận để solver chứng minh vô nghiệm. inputs/examples.json gồm hai bài có nghiệm và một bài Akari vô nghiệm.

7. Kiến trúc và tương tác

logic_lab/games chứa luật; search.py quản lý frontier, ngân sách, thống kê và trace; service.py xử lý solve/check/hint; http_api.py dùng chung cho server local và WSGI. Giao diện không chứa bộ tìm kiếm riêng. JavaScript có kiểm tra nhẹ để phản hồi tức thì, nhưng Python xác nhận hoàn thành.

Khi xin gợi ý, các lựa chọn đã đánh dấu được giữ làm ràng buộc. Nếu có nghiệm, trả một hành động tiến tới nghiệm đó. Nếu nhánh hiện tại đã được chứng minh vô nghiệm, solver giải bài gốc rồi đề xuất bỏ đèn, bỏ dấu × hoặc sửa hướng một ô khác nghiệm. Một bước sửa có thể chưa đủ và không được gọi là cách sửa ít bước nhất. Máy chỉ đổi bảng khi người chơi bấm Áp dụng.

Demo có phạm vi giữ nguyên các lựa chọn hoặc cho phép thay đổi chúng. Trace là lịch sử lấy trạng thái ra xét, không phải đường đi của một nghiệm. Hai trạng thái liên tiếp có thể thuộc hai nhánh khác nhau. Demo lưu tối đa 350 trạng thái; nếu bị cắt, vẫn có nút xem nghiệm cuối. Với Pipes, ống mờ chỉ hướng minh họa của ô chưa gán.

Mỗi yêu cầu dùng bản chụp bảng. Giao diện hủy chờ và bỏ phản hồi cũ khi đổi bảng, thuật toán hoặc đóng demo. HTTP chỉ cho tối đa 2 lần tìm kiếm đồng thời trong một tiến trình. Đồng hồ và lịch sử độc lập với thời gian solver; sửa hoặc hoàn tác không làm thời gian chơi lùi lại. Bảng đã hoàn thành được xác nhận lại khi khôi phục nhưng không lặp hộp thoại chúc mừng.

8. Phương pháp thực nghiệm

Lệnh đo chính: python benchmark.py --repeat 7 --suite extended --output results/benchmark.json.

Máy đo dùng Python 3.12.14, Windows 11, kiến trúc Intel64 Family 6 Model 186 Stepping 2. Bộ dữ liệu, nền tảng, revision Git, SHA-256 mã nguồn/input và toàn bộ mẫu thô được lưu trong JSON. Revision được ghi nhận là có thay đổi chưa commit, nên mã băm nguồn là dấu vết chính xác hơn tên commit đơn lẻ.

Mỗi tổ hợp chạy một lần warm-up rồi đo 7 lần bằng perf_counter; báo cáo median, lưu cả min/max và từng mẫu. Đo bộ nhớ ở lần chạy riêng bằng tracemalloc sau gc.collect, tránh để profiling làm sai thời gian được công bố. Ngân sách thời gian thường là 4 giây; lần đo bộ nhớ là 30 giây do profiling chậm hơn. Cả hai có cùng giới hạn 100000 trạng thái được lấy ra xét.

Phạm vi đo là search: bao gồm analyze gốc, sinh con, kiểm tra ràng buộc, heuristic/heap và kiểm tra đích trong search; không gồm khởi tạo bài toán, HTTP, vẽ bảng và lưu trace. Peak bộ nhớ là cấp phát Python được tracemalloc theo dõi trong search, đơn vị KiB = 1024 byte; không phải RSS và không gồm mọi cấu trúc được tạo trước đó. Vì vậy số liệu không đại diện tổng RAM khi chạy giao diện.

expanded đếm mọi trạng thái lấy ra xét, gồm cả đích. generated đếm gốc và mọi con được sinh, kể cả bị loại. pruned đếm gốc/con bị analyze loại. frontier_peak đếm số node chờ lớn nhất; nhánh bị loại ngay từ gốc có frontier_peak bằng 0. Lời giải được kiểm tra sau mọi lần chạy; việc kiểm tra sau chạy không thuộc thời gian search.

Các input được chạy tuần tự theo thứ tự xác định, DFS trước Greedy cho từng input; không chạy tải khác có chủ ý đồng thời. Đây là đo trên một môi trường và bộ input có cấu trúc tương tự nhau, không phải khảo sát thống kê rộng hay phép đo end-to-end. Chênh lệch nhỏ cỡ phần nghìn mili giây cần được đọc cùng nhiễu hệ thống.

9. Kết quả và giải thích

Tất cả 52 tổ hợp thuật toán/input hoàn thành 7/7 lần đo thời gian và lần đo bộ nhớ. Không có timeout trong bộ đo này. Bảng đầy đủ được đính kèm ở phụ lục thực nghiệm; dữ liệu gốc có thể mở trong JSON hoặc CSV.

9.1. Pipes

Trên 13 input Pipes, tổng số trạng thái DFS xét là 453, Greedy là 455. Tổng median theo từng input lần lượt 49.331 ms và 53.001 ms. Đại lượng tổng median chỉ mô tả tải của bộ input này; không phải median của một phiên chạy ghép.

Nhiều màn nhỏ chỉ có một hướng khả thi sau kiểm tra biên và hàng xóm. Hai thuật toán đi qua gần như cùng các trạng thái, nên heuristic ít tạo lợi thế. Greedy vẫn tính h và quản lý heap; chênh lệch nhỏ về thời gian không thể dùng để khẳng định lợi thế tổng quát.

Với pipes-8-seed-103, DFS xét 67 trạng thái, frontier lớn nhất 4; Greedy xét 69, frontier lớn nhất 3. Median tương ứng 12.033 và 13.000 ms. Peak cấp phát Python tương ứng 213.34 và 210.48 KiB. Frontier ít hơn không tự động đồng nghĩa tổng peak bộ nhớ hoặc thời gian ít hơn theo cùng một tỉ lệ.

9.2. Light Up

Trên 13 input Light Up, tổng số trạng thái DFS xét là 1945, Greedy là 468. Tổng median từng input lần lượt 57.497 ms và 15.350 ms. Khác biệt tổng hợp chịu ảnh hưởng mạnh của một vài input, đặc biệt seed 102 kích thước 8x8.

Ở lightup-8-seed-102, DFS xét 1584 trạng thái, median 47.455 ms, peak 366.38 KiB; Greedy xét 68 trạng thái, median 2.577 ms, peak 38.19 KiB. Trong trường hợp này, ưu tiên vùng sáng và chỉ tiêu đèn giúp tránh nhiều nhánh mà thứ tự thử 0 trước của DFS khám phá.

Ngược lại, lightup-7 có DFS xét 51 trạng thái trong 1.629 ms, Greedy xét 142 trong 4.711 ms. Peak tương ứng 35.10 và 61.02 KiB. Điểm h không biểu diễn đầy đủ tương tác ràng buộc tương lai; h nhỏ có thể dẫn tới nhiều nhánh thất bại, còn DFS tình cờ gặp nhánh tốt theo thứ tự cố định. Đây là phản ví dụ cụ thể cho nhận định Greedy luôn thắng.

9.3. Diễn giải thận trọng

Kích thước lớn hơn không bảo đảm khó hơn: cấu trúc tường, số và hướng ống quyết định mức cắt nhánh. Một nghiệm chứng kiến không chứng minh tính duy nhất. Một màn có nghiệm không đảm bảo mọi lựa chọn người chơi còn nghiệm. Kết quả đo của bộ sinh hiện tại không đủ để suy rộng sang mọi màn trên website gốc hoặc kết luận thuật toán tối ưu.

10. Kiểm chứng tính đúng và giao diện

Bộ kiểm thử Python gồm 47 test ở thời điểm phát hành. Có oracle độc lập kiểm tra bảng hoàn chỉnh; các bảng nhỏ được liệt kê toàn bộ nghiệm rồi so kết luận có/vô nghiệm với search trên nhiều trạng thái có ràng buộc. Bộ mở rộng được kiểm tra cả hai thuật toán bằng oracle độc lập. Cây tìm kiếm nhân tạo kiểm tra DFS không dùng h, Greedy theo h, phá hòa ổn định, thống kê và phân biệt limit/unsat.

Các kịch bản chơi gồm 48 mẫu nước sai có seed cố định trên 8 màn, 2 thuật toán; liên tiếp áp dụng gợi ý/sửa đều hoàn thành. Test HTTP chạy server thật và xác minh cả tài nguyên web lẫn API. API được thử dữ liệu sai kiểu, JSON lỗi/lồng quá sâu, nguồn yêu cầu sai và khi hết slot tìm kiếm.

JavaScript có 800 đối chiếu kết quả kiểm tra luật với Python, kiểm tra lịch sử, đồng hồ, timeout/hủy yêu cầu, phản hồi đến sai thứ tự và xác nhận thắng. Kiểm tra trình duyệt Chromium thật chạy 16 demo, dùng bàn phím, sửa đèn sai, áp dụng gợi ý tới thắng, khôi phục tiến độ và bố cục ở độ rộng 1440, 768, 390, 320 px. Ảnh giao diện được xem lại; ô số không bị làm mờ bởi trạng thái button disabled.

Những kiểm tra này cung cấp bằng chứng cho các kịch bản đã chạy, không phải chứng minh phần mềm không còn lỗi. Chưa có xác minh trên Safari/Firefox, cảm ứng vật lý hoặc khảo sát người dùng. Khi thay mã, cần chạy lại các lệnh và checklist giao diện.

11. Hướng dẫn chạy và tái lập

```
python app.py
python -m logic_lab --level pipes-5 --algorithm dfs --trace 20
python -m logic_lab --input inputs/examples.json --algorithm greedy
python -m unittest discover -s tests -v
node tests/check_frontend.mjs
python benchmark.py --repeat 7 --suite extended --output results/benchmark.json
```

Mở http://127.0.0.1:8765 khi chạy giao diện. Không cần thư viện ngoài cho ứng dụng, CLI, unit test và benchmark. Với Python nằm ngoài PATH, đặt biến môi trường PYTHON là đường dẫn trình thông dịch trước khi chạy kiểm tra JavaScript. CLI trả mã 2 nếu có kết quả limit; kết quả unsat được coi là một lần giải đã kết luận.

Kiểm tra Chromium là tùy chọn, cần Playwright và Chromium đã cài. Tạo lại báo cáo PDF cần ReportLab và font Unicode; tạo lại PowerPoint dùng Artifact Tool trong môi trường Codex. Những thư viện tạo tài liệu không cần cho người chấm chạy bài.

12. Giới hạn và hướng cải tiến

Thực nghiệm hiện tại dựa trên bộ sinh có nghiệm và input nhỏ/vừa, có thể có nhiều nghiệm. DFS và Greedy dùng cắt nhánh chung để so chính sách chọn trạng thái; kết quả không đại diện một DFS không kiểm tra ràng buộc. Chưa tối ưu số thao tác, số đèn, số nước sửa hoặc bộ nhớ theo RSS. Gợi ý sửa giữ tính minh bạch nhưng có thể cần nhiều lần.

Các hướng tiếp theo gồm đo trên bộ input đa dạng hơn, trường hợp khó/vô nghiệm có kiểm soát, nhiều seed hơn và nhiều nền tảng; so heuristic bằng ablation; tối ưu ít nước sửa bằng bài toán có trọng số; thêm ghim/bỏ ghim ô Pipes; giảm chi phí vùng nhìn thấy Light Up bằng bitset. Các hướng này được tách khỏi kết quả đã hiện thực và đo.

13. Kết luận

Dự án hiện thực đầy đủ đường đi từ mô hình hai Logic Puzzles đến DFS/Greedy bằng Python, demo trực quan, gợi ý theo trạng thái và thực nghiệm có dữ liệu thô. Trên bộ 26 input đã kiểm chứng, cả hai thuật toán tìm được nghiệm hợp lệ. Heuristic tạo khác biệt rõ ở một số Light Up, trong khi Pipes chịu ảnh hưởng mạnh của cắt nhánh cục bộ. Các kết luận được giới hạn ở phạm vi input, cấu hình và cách đo đã công bố.

Tài liệu tham khảo

[1] Pipes, luật chơi chính thức: https://www.puzzle-pipes.com/ (đối chiếu ngày 25/10/2026).

[2] Light Up, luật chơi chính thức: https://www.puzzle-light-up.com/ (đối chiếu ngày 25/10/2026).

[3] Đề BTL1 do người dùng cung cấp: Bài tập lớn 1, học kỳ 1 năm học 2026-2027, chủ đề Tìm kiếm, 2 trang. Tài liệu này là căn cứ yêu cầu; không đưa bản sao đề vào gói mã nguồn.

[4] Mã nguồn và thực nghiệm của dự án: logic_lab/, tests/, results/benchmark.json, results/benchmark.csv; các phần mô hình và giả mã trong báo cáo mô tả trực tiếp bản cài đặt này.

Phụ lục thực nghiệm

Nguồn: results/benchmark.json và results/benchmark.csv.

Python 3.12.14; Windows-11-10.0.26200-SP0. 7 lần đo thời gian mỗi tổ hợp; một lần đo bộ nhớ riêng.

ms = median thời gian search; KiB = peak cấp phát Python trong search, không phải RSS. N = trạng thái đã xét, gồm đích.

Ngân sách: 4.0 giây/lượt thường; 30.0 giây/lượt đo bộ nhớ; 100000 trạng thái/lượt.

Nếu một tổ hợp chưa hoàn thành mọi lượt, bảng ghi rõ trạng thái; không diễn giải median của lượt bị giới hạn như thời gian giải thành công.

Pipes

Input	DFS ms	Greedy ms	DFS KiB	Greedy KiB	DFS N	Greedy N
pipes-3	0.253	0.265	9.84	9.73	10	10
pipes-4	0.735	0.754	20.76	20.80	17	17
pipes-5	1.885	1.864	46.08	46.16	28	28
pipes-6	3.631	3.853	84.48	84.52	37	37
pipes-4-seed-101	0.648	0.674	21.57	21.61	17	17
pipes-4-seed-102	0.650	0.668	21.89	21.93	17	17
pipes-4-seed-103	0.657	0.683	22.31	22.35	17	17
pipes-6-seed-101	3.018	3.096	83.05	83.09	37	37
pipes-6-seed-102	2.952	3.087	83.20	83.23	37	37
pipes-6-seed-103	3.023	3.306	83.45	83.48	37	37
pipes-8-seed-101	10.332	10.561	195.39	195.45	67	67
pipes-8-seed-102	9.514	11.190	209.72	209.76	65	65
pipes-8-seed-103	12.033	13.000	213.34	210.48	67	69

Light Up

Input	DFS ms	Greedy ms	DFS KiB	Greedy KiB	DFS N	Greedy N
lightup-3	0.064	0.087	4.74	6.23	7	9
lightup-5	0.625	0.780	14.82	18.55	32	35
lightup-6	0.447	0.713	12.41	14.36	20	26
lightup-7	1.629	4.711	35.10	61.02	51	142
lightup-4-seed-101	0.152	0.184	5.39	5.62	9	9
lightup-4-seed-102	0.162	0.161	7.34	8.69	14	11
lightup-4-seed-103	0.119	0.142	5.24	5.49	7	7
lightup-6-seed-101	0.686	0.608	19.15	19.86	29	23
lightup-6-seed-102	0.683	0.802	15.56	18.84	31	29
lightup-6-seed-103	1.397	0.620	21.94	17.55	56	22
lightup-8-seed-101	2.197	2.221	21.59	26.36	52	44
lightup-8-seed-102	47.455	2.577	366.38	38.19	1584	68
lightup-8-seed-103	1.881	1.744	31.78	31.41	53	43

Dấu vết tái lập

Thời điểm UTC: 2026-10-25T13:38:49.795438+00:00.

SHA-256 input: a71abd6a65c3bed8f523a15c4636f54ed6f1db8fd242bcbbe5383c709d5fe3d7.

SHA-256 nguồn benchmark/logic_lab: 099ca1fab095061fcedca7bcf604be403df7764d23fc073f5c9b8385587ca78b.

JSON lưu nguyên input, từng thời gian, thống kê từng lượt, trạng thái kiểm tra nghiệm và thông tin môi trường. Để chạy lại nguyên bộ input đã lưu, xuất trường inputs ra một tệp JSON rồi truyền --input cho benchmark.py.